

Bypass de OTP/MFA por manipulacion de parametros

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El segundo factor (OTP, código por email/SMS) debe verificarlo y recordarlo EL SERVIDOR. El bypass aparece cuando el backend acepta del cliente una señal de 'ya verificado' (un campo `is_verified`, `verified`, `mfa_passed`, un paso de estado) y la respeta aunque el OTP sea incorrecto. Es `parameter tampering` aplicado a un control de seguridad: `ver client-side-controls`.

El desarrollador reutiliza un formulario o un JSON donde el estado de verificación viaja junto a los datos y confía en él. Como el cliente controla todo lo que envía, puede poner `is_verified=true`. La causa raíz es mantener el estado de seguridad en el cliente en vez de derivarlo en el servidor tras comprobar el OTP.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier paso de 2FA/OTP tras un login válido: una pantalla de 'introduce el código', un endpoint `verify_otp`, un formulario con campos ocultos de estado. Intercepta la petición y mira que campos viajan además del propio código.

Cómo identificarla

- La petición de OTP incluye, además del código, un campo como `is_verified`, `verified`, `status` o `step`.
- Campos ocultos de estado de verificación en el HTML del formulario.
- No hay `rate limiting` ni invalidación del código tras varios intentos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Envía el OTP (aunque sea incorrecto) añadiendo un campo de estado tipo `is_verified=true` en el cuerpo y comprueba si te deja pasar.

Evidencia esperada: Accedes al recurso protegido (dashboard) sin un OTP válido, porque el servidor aceptó tu flag de verificación.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s -b cookies.txt -F 'otp=000000' -F 'is_verified=true' "$URL/verify_otp.php"
```

Objetivo y contexto: Envía el OTP (incorrecto a propósito) añadiendo el campo de estado como sección multipart (-F). Si el servidor confía en is_verified, pasa el 2FA sin el código real.

Resultado esperado: Una redirección o acceso al dashboard pese al OTP inválido = bypass confirmado. Si exige el código correcto pese al campo, el estado se valida server-side.

Anatomía de los payloads

La primera prueba es `curl -s -b cookies.txt -F 'otp=000000' -F 'is_verified=true' "$URL/verify_otp.php"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Accedes al recurso protegido (dashboard) sin un OTP válido, porque el servidor aceptó tu flag de verificación.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Llega al paso de OTP con una sesión válida y captura la petición de verificación.
- Localiza campos de estado además del código; si el cuerpo es multipart/form-data, cada campo va como una sección propia (un parámetro añadido como cabecera HTTP NO aparece en \$_POST).
- Reenvía el OTP con is_verified=true (o el nombre que veas) y observa si concede acceso.
- Si funciona, documenta que el estado de verificación se confiaba al cliente; guarda la cookie de sesión para el siguiente paso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor recalcula la verificación en cada petición contra su propio estado y no lee ninguna flag de verificación del cliente.

- El OTP se valida server-side, es de un solo uso, caduca y esta atado a usuario+sesion, sin ningun parametro de estado manipulable.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El servidor confía en un dato que envía el cliente (is_verified=true) para dar por superado el 2FA; tu decides el estado de una comprobación que debería calcular solo el servidor. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(mfa-otp-bypass\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.